| Status | Started | Completed |
|---|---|---|
| Finished | Wednesday, 11 November 2026, 1:10 PM | Wednesday, 11 November 2026, 1:11 PM |

| Duration | Grade |
|---|---|
| 1 min 4 secs | **20.00** out of 20.00 (**100**%) |

# Question 1

Flag question

Correct Mark 1.00 out of 1.00

Your batch job processes 8,000 documents per night. After the batch, 640 documents fail because they are scanned PDFs that exceed the context limit when OCR text is included. You need to reprocess these 640 efficiently. What is the correct strategy?

a. Identify the 640 failed documents by their custom_id values from the batch results. Chunk each large OCR document into overlapping sections and resubmit only those 640 as a new batch with the chunking approach. The 7,360 successful documents are not reprocessed.

b. Resubmit all 8,000 documents with a shorter prompt to reduce overall token usage.

c. Accept the 640 failures and reduce the nightly batch size to avoid future failures.

d. Switch the 640 failures to the synchronous API for immediate reprocessing.

The correct answer is: Identify the 640 failed documents by their custom_id values from the batch results. Chunk each large OCR document into overlapping sections and resubmit only those 640 as a new batch with the chunking approach. The 7,360 successful documents are not reprocessed.

## Question 2

Flag question

Correct Mark 1.00 out of 1.00

Your batch pipeline failed for 500 documents because the batch request was submitted but you forgot to include the custom_id field in the request bodies. The batch completed with 500 results but you cannot match results to documents. What does this reveal about custom_id usage?

a. The batch API automatically assigns sequential IDs when custom_id is omitted.
b. Resubmit all 500 documents — the API supports re-running failed batches.
c. custom_id is essential for batch workflows: without it, correlating results to source documents is unreliable since batch results may not preserve submission order. The 500 results are essentially orphaned. Always include a unique custom_id (e.g. document hash or database ID) in every batch request to enable reliable correlation and failure identification. ✓
d. custom_id is optional — you can match by response order since batches preserve order.

The correct answer is: custom_id is essential for batch workflows: without it, correlating results to source documents is unreliable since batch results may not preserve submission order. The 500 results are essentially orphaned. Always include a unique custom_id (e.g. document hash or database ID) in every batch request to enable reliable correlation and failure identification.

## Question 3

Flag question

Correct Mark 1.00 out of 1.00

Your data extraction pipeline processes annual reports from public companies. Financial figures appear in different scales across documents: some use thousands ('$1,500' meaning $1.5M), others use millions, others use full numbers. Downstream systems require all values in full dollars. What extraction strategy handles the scale ambiguity?

a. Create separate extraction schemas for each scale convention.
b. Add a scale_convention field to the schema and handle conversion downstream.
c. Extract all numbers as-is and apply scale detection in post-processing.
d. Design the extraction prompt to identify and normalise the document's reporting scale: instruct the model to first detect the scale convention (thousands/millions/full) from document context (e.g. '(in thousands)' header) and then normalise all extracted values to full dollars. Include few-shot examples for each scale convention. ✓

The correct answer is: Design the extraction prompt to identify and normalise the document's reporting scale: instruct the model to first detect the scale convention (thousands/millions/full) from document context (e.g. '(in thousands)' header) and then normalise all extracted values to full dollars. Include few-shot examples for each scale convention.

## Question 4

Flag question

Correct Mark 1.00 out of 1.00

Your classification agent outputs confidence scores of 0.9+ for all documents regardless of actual classification difficulty. This makes it impossible to identify documents that genuinely need human review. What extraction schema design improves confidence discrimination?

a. Add a rationale field alongside confidence_score where the model must explain its reasoning for the assigned confidence. When the model must articulate why it is confident, it naturally produces more calibrated scores and surfaces genuine uncertainty through less convincing rationale text that review routing can act on.
b. Remove confidence scores and use only binary high/low routing.
c. Instruct the model to 'use the full range of confidence scores from 0.0 to 1.0.'
d. Set temperature to 0.5 to produce more varied confidence scores.

The correct answer is: Add a rationale field alongside confidence_score where the model must explain its reasoning for the assigned confidence. When the model must articulate why it is confident, it naturally produces more calibrated scores and surfaces genuine uncertainty through less convincing rationale text that review routing can act on.

## Question 5

Flag question

Correct Mark 1.00 out of 1.00

Your pipeline batch processes 12,000 customer surveys overnight. After the batch, 900 surveys (7.5%) fail because they contain languages the extraction model does not support well. How do you handle these 900 efficiently?

a. Identify the 900 failed surveys by their custom_id values. For these specific surveys, either: translate them to a supported language first and resubmit, or route them to a specialist model or human review. Reprocessing only the 900 failures preserves the 11,100 successful extractions.
b. Resubmit all 12,000 surveys with a multilingual extraction prompt.
c. Add more languages to the extraction schema's language enum.
d. Accept the 7.5% failure rate as inherent to multilingual extraction.

The correct answer is: Identify the 900 failed surveys by their custom_id values. For these specific surveys, either: translate them to a supported language first and resubmit, or route them to a specialist model or human review. Reprocessing only the 900 failures preserves the 11,100 successful extractions.

## Question 6

Flag question

Correct Mark 1.00 out of 1.00

Your extraction pipeline processes vendor invoices where the discount field is present in 40% of invoices and absent in 60%. The schema marks discount as required. Claude fabricates discount values for the 60% without discounts. What schema change eliminates fabrication?

a. Change discount to type: string so 'N/A' can be used as a null substitute.
b. Mark discount as optional (nullable): type: ['number', 'null']. This gives the model a legitimate way to express the genuine absence of a discount rather than being forced to fabricate a value to satisfy a required field constraint. ✓
c. Add a note: 'Use 0 for discount when no discount is present.'
d. Add an enum constraint: [0, 5, 10, 15, 20] with 0 as the default.

The correct answer is: Mark discount as optional (nullable): type: ['number', 'null']. This gives the model a legitimate way to express the genuine absence of a discount rather than being forced to fabricate a value to satisfy a required field constraint.

## Question 7

Flag question

Correct Mark 1.00 out of 1.00

Your pipeline must handle documents where the total_amount field represents either the pre-tax total or post-tax total depending on the vendor. The schema has a single total_amount field without disambiguation. Downstream systems use this field incorrectly 20% of the time. What schema improvement resolves this ambiguity?

a. Create two separate total fields: pretax_total and posttax_total, both optional.
b. Add an instruction: 'Clarify whether total_amount is pre- or post-tax.'
c. Accept the 20% error rate — tax calculation is downstream's responsibility.
d. Add a total_type enum field: {enum: ['pre_tax', 'post_tax', 'unknown']} alongside total_amount. This makes the tax-inclusion status machine-readable and explicit, enabling downstream systems to handle each case correctly without parsing ambiguous descriptions. ✓

The correct answer is: Add a total_type enum field: {enum: ['pre_tax', 'post_tax', 'unknown']} alongside total_amount. This makes the tax-inclusion status machine-readable and explicit, enabling downstream systems to handle each case correctly without parsing ambiguous descriptions.

## Question 8

Flag question

Correct Mark 1.00 out of 1.00

Your extraction pipeline uses few-shot examples from documents collected 18 months ago. The document format has since evolved — new required fields, changed naming conventions, different table structures. Extraction accuracy has declined from 94% to 81% over the past 6 months. What is the most direct fix?

a. Update the few-shot examples to use recent documents that reflect the current format. Stale examples that do not match current document structure actively mislead the model. Replace examples with current representative samples covering the new fields, naming conventions, and table structures.

b. Increase max_tokens to allow the model to handle the new format complexity.

c. Switch to a more capable model that can handle format evolution better.

d. Add more examples from the old format to improve pattern recognition.

The correct answer is: Update the few-shot examples to use recent documents that reflect the current format. Stale examples that do not match current document structure actively mislead the model. Replace examples with current representative samples covering the new fields, naming conventions, and table structures.

## Question 9

Flag question

Correct Mark 1.00 out of 1.00

Your review pipeline produces findings with a detected_pattern field that records which code pattern triggered the finding. After 3 months, analysis shows that 40% of dismissed findings share 5 specific detected_pattern values. What does this tell you and what action should you take?

a. The detected_pattern data reveals systematic false positives: those 5 patterns are producing findings that developers consistently reject as non-issues. Update the review criteria to explicitly exclude or deprioritise those 5 patterns, reducing noise and restoring developer trust in the review system.

b. Remove the detected_pattern field — it is not adding value.

c. Dismiss rates are random — this pattern analysis is not meaningful.

d. Lower the severity of findings associated with those 5 patterns.

The correct answer is: The detected_pattern data reveals systematic false positives: those 5 patterns are producing findings that developers consistently reject as non-issues. Update the review criteria to explicitly exclude or deprioritise those 5 patterns, reducing noise and restoring developer trust in the review system.

## Question 10

Flag question

Correct Mark 1.00 out of 1.00

Your code review prompt says 'check this PR for correctness.' After analysis, 50% of findings are correct, 30% are style preferences, and 20% are genuine bugs. The goal is to surface only genuine bugs. What prompting change most directly achieves this?

a. Define what constitutes a genuine bug explicitly: 'Report only: logic errors that produce incorrect outputs, conditions that cause runtime exceptions, security vulnerabilities, and missing error handling for expected failure cases. Do not report: style, naming, refactoring opportunities, performance suggestions, or code organisation preferences.' ✓

b. Add 'ignore style issues' to the existing prompt.

c. Change to 'check for bugs' — more focused instruction.

d. Use a two-pass approach where the first pass identifies all issues and the second pass filters to bugs only.

The correct answer is: Define what constitutes a genuine bug explicitly: 'Report only: logic errors that produce incorrect outputs, conditions that cause runtime exceptions, security vulnerabilities, and missing error handling for expected failure cases. Do not report: style, naming, refactoring opportunities, performance suggestions, or code organisation preferences.'

## Question 11

Flag question

Correct Mark 1.00 out of 1.00

Your pipeline extracts financial data from documents. Amounts appear as '$1,500', '1500 USD', '€1.500,00', and 'fifteen hundred dollars.' A regex post-processor correctly handles the first two formats but fails on the last two. What extraction strategy handles all four formats reliably?

a. Reject documents containing non-standard amount formats.

b. Create separate extraction schemas for each currency format type.

c. Instruct the model to normalise all monetary amounts to a canonical float value in the base currency during extraction. Include few-shot examples showing each format being normalised: '$1,500' → 1500.0, '€1.500,00' → 1500.0, 'fifteen hundred dollars' → 1500.0. The model's semantic understanding handles informal expressions that regex cannot. ✓

d. Extract amounts as raw strings and post-process all formats in application code.

The correct answer is: Instruct the model to normalise all monetary amounts to a canonical float value in the base currency during extraction. Include few-shot examples showing each format being normalised: '$1,500' → 1500.0, '€1.500,00' → 1500.0, 'fifteen hundred dollars' → 1500.0. The model's semantic understanding handles informal expressions that regex cannot.

## Question 12

Flag question

Correct Mark 1.00 out of 1.00

Your document extraction pipeline processes invoices from 50 different vendors. For 3 specific vendors, extraction accuracy is 55% even though overall accuracy is 94%. These 3 vendors use unusual table formats for line items. What is the most targeted fix?

a. Add more general extraction examples to improve overall accuracy.
b. Flag all documents from the 3 vendors for manual review.
c. Switch to a more capable model to handle unusual table formats.
d. Add 2-3 vendor-specific few-shot examples showing correct extraction from each of the 3 problematic vendors' unusual table formats. These targeted examples teach the model the specific structural patterns of those vendors without requiring system-wide changes. ✓

The correct answer is: Add 2-3 vendor-specific few-shot examples showing correct extraction from each of the 3 problematic vendors' unusual table formats. These targeted examples teach the model the specific structural patterns of those vendors without requiring system-wide changes.

## Question 13

Flag question

Correct Mark 1.00 out of 1.00

Your code generation pipeline produces correct code 88% of the time. For a production deployment, you need 96% accuracy. Simply generating more examples with better prompts has plateaued at 88-89%. What architectural change breaks this plateau?

a. Use a more capable model tier to raise the accuracy ceiling.
b. Increase max_tokens significantly to allow more careful generation.
c. Generate code at a lower temperature for more conservative, predictable output.
d. Implement a generate-then-critique architecture: generate code in the first pass, then run a second independent critique pass that evaluates the code against explicit correctness criteria, provides specific feedback, and generates a corrected version. Independent review consistently surfaces issues that single-pass generation misses. ✓

The correct answer is: Implement a generate-then-critique architecture: generate code in the first pass, then run a second independent critique pass that evaluates the code against explicit correctness criteria, provides specific feedback, and generates a corrected version. Independent review consistently surfaces issues that single-pass generation misses.

## Question 14

Flag question

Correct Mark 1.00 out of 1.00

Your pipeline creates a two-step extraction: first extract metadata (document_type, date, parties), then use the metadata to guide a deeper extraction pass. The second pass sometimes runs before the first completes, producing poor results. What is the deterministic fix?

a. Add extract_metadata as an automatic pre-step inside the detail extraction tool.
b. Add a system prompt instruction: 'Always complete metadata extraction before the detailed pass.'
c. Use tool_choice: {type: 'any'} to let the model determine the optimal order.
d. Implement the sequencing in application code: make the first API call with tool_choice forced to the metadata extraction tool. Only after receiving and validating the metadata programmatically does the application make the second API call for detailed extraction, passing the metadata in context. Programmatic sequencing eliminates probabilistic ordering.

The correct answer is: Implement the sequencing in application code: make the first API call with tool_choice forced to the metadata extraction tool. Only after receiving and validating the metadata programmatically does the application make the second API call for detailed extraction, passing the metadata in context. Programmatic sequencing eliminates probabilistic ordering.

## Question 15

Flag question

Correct Mark 1.00 out of 1.00

Your extraction schema defines confidence_score as a float for each extracted field. After deployment, all confidence scores cluster between 0.87 and 0.93 regardless of actual extraction difficulty. Human review finds that scores of 0.91 correspond to correct extractions only 78% of the time. What is needed before using confidence for routing?

a. Raise the confidence threshold to 0.99 to reduce human review workload.
b. Accept the clustered scores — the model's confidence calibration is inherently limited.
c. Calibrate confidence thresholds using a labeled validation set: measure the actual extraction accuracy at each reported confidence band (0.85–0.90, 0.90–0.95, etc.) and set human review routing thresholds based on the observed real-world accuracy, not on the raw model scores. A score of 0.91 may warrant review if it corresponds to 78% accuracy.
d. Instruct the model to use the full 0.0–1.0 scale more evenly.

The correct answer is: Calibrate confidence thresholds using a labeled validation set: measure the actual extraction accuracy at each reported confidence band (0.85–0.90, 0.90–0.95, etc.) and set human review routing thresholds based on the observed real-world accuracy, not on the raw model scores. A score of 0.91 may warrant review if it corresponds to 78% accuracy.

## Question 16

Flag question

Correct Mark 1.00 out of 1.00

Your code review pipeline prompt currently says: 'Report any code quality issues you find.' After analysis, you discover that 40% of reported findings are related to minor formatting preferences rather than substantive issues. What is the most precise fix?

a. Create separate prompts for different issue categories.
b. Rewrite the instruction with categorical precision: 'Report findings in these categories only: potential runtime errors, security vulnerabilities, missing error handling, incorrect logic, and missing null checks. Do not report: naming preferences, formatting style, whitespace, comment style, or non-breaking code organisation choices.' ✓
c. Add 'focus on serious issues only' to the prompt.
d. Add 'ignore minor issues' to the prompt.

The correct answer is: Rewrite the instruction with categorical precision: 'Report findings in these categories only: potential runtime errors, security vulnerabilities, missing error handling, incorrect logic, and missing null checks. Do not report: naming preferences, formatting style, whitespace, comment style, or non-breaking code organisation choices.'

## Question 17

Flag question

Correct Mark 1.00 out of 1.00

Your validation retry loop sends: 'There was a validation error. Please try again.' The retry succeeds in only 15% of cases. What change to the retry message produces the highest improvement in retry success rate?

a. Switch to a more capable model for retry attempts.
b. Increase retry count from 3 to 10.
c. Include specific error details in the retry prompt: the field that failed, the exact value that was extracted, the rule it violated, and a correctly formatted example. For example: 'The invoice_date field contained 15/03/2024 but requires ISO 8601 format (YYYY-MM-DD). Example correct value: 2024-03-15. Please re-extract and correct this field.' ✓
d. Add 'This is very important — please be careful this time' to the retry message.

The correct answer is: Include specific error details in the retry prompt: the field that failed, the exact value that was extracted, the rule it violated, and a correctly formatted example. For example: 'The invoice_date field contained 15/03/2024 but requires ISO 8601 format (YYYY-MM-DD). Example correct value: 2024-03-15. Please re-extract and correct this field.'

## Question 18

Flag question

Correct Mark 1.00 out of 1.00

Your prompts for code generation have improved significantly and produce correct code 90% of the time. The remaining 10% have subtle logical errors that pass static analysis and basic unit tests. What approach improves detection of subtle logical errors in generated code?

a. Lower temperature to 0 for more conservative, less error-prone generation.
b. Increase max_tokens to give the model more reasoning capacity.
c. Implement a multi-pass architecture: generate code in the first pass, then in a second independent pass ask the model to reason through the logic step-by-step looking for cases where the code might produce incorrect results. An independent reviewer pass without the generation context is more effective at detecting logical issues. ✓
d. Run the code generation three times and use the majority consensus output.

The correct answer is: Implement a multi-pass architecture: generate code in the first pass, then in a second independent pass ask the model to reason through the logic step-by-step looking for cases where the code might produce incorrect results. An independent reviewer pass without the generation context is more effective at detecting logical issues.

## Question 19

Flag question

Correct Mark 1.00 out of 1.00

Your pipeline processes academic papers and must extract methodology sections. Some papers have clearly labeled 'Methodology' sections; others describe their methods throughout the paper without a dedicated section. The current extraction correctly handles labeled sections but misses embedded methods. What prompt change addresses this?

a. Add 'also look for methods described outside the Methodology section' to the prompt.
b. Change the extraction instruction from 'extract the Methodology section' to 'identify and extract all content that describes how the study was conducted, what methods were used, and how data was collected or analysed — regardless of whether this appears in a dedicated section or is distributed throughout the paper.' Focus on semantic content rather than structural location. ✓
c. Create a secondary extraction pass specifically for papers without methodology sections.
d. Only extract from explicitly labeled methodology sections.

The correct answer is: Change the extraction instruction from 'extract the Methodology section' to 'identify and extract all content that describes how the study was conducted, what methods were used, and how data was collected or analysed — regardless of whether this appears in a dedicated section or is distributed throughout the paper.' Focus on semantic content rather than structural location.

# Question 20

Flag question

Correct Mark 1.00 out of 1.00

Your pipeline extracts 'termination_for_cause' clauses from contracts. In 15% of contracts, this clause is implicit (the contract describes what constitutes breach but never uses the phrase 'termination for cause'). The model currently marks these as null because the explicit phrase is absent. What extraction approach handles implicit clauses?

a. Rewrite the extraction instruction to focus on the semantic concept rather than surface text: 'Identify any clause that addresses the conditions under which the contract can be terminated due to one party's failure to perform, regardless of the specific terminology used. Include implicit termination-for-cause provisions even when that exact phrase is not present.'

b. Accept null for contracts without explicit termination language.

c. Add 'look for implied termination rights' to the extraction prompt.

d. Create a separate implicit_termination_for_cause boolean field.

The correct answer is: Rewrite the extraction instruction to focus on the semantic concept rather than surface text: 'Identify any clause that addresses the conditions under which the contract can be terminated due to one party's failure to perform, regardless of the specific terminology used. Include implicit termination-for-cause provisions even when that exact phrase is not present.'

Previous activity
Claude Code Configuration & Workflows - Practice Assessment

Jump to...

Next activity
Context Management & Reliability - Practice Assessment